

# Transformer Riffage

**Vishakh Begari**

*Independent Researcher*

*Gauting, 82131, Germany*

DJENTLEVIBE32168@GMAIL.COM

**Editor:** Adam Smasher

## Abstract

Ever since the electrification of guitar, there has been an explosion in rhythmic complexity, tone experimentation and playing techniques. Music generation using Transformers Huang et al. (2018) has generally been used for creating mostly musical notes. In this paper, a new approach is proposed to capture the diverse expressions commonly featured in modern metal music. A new tokenisation method encapsulates the various guitar techniques and a sequence level attention component which tries to capture the segmentation between different musical phrases. An inference method is implemented that provides meaningful validation without human evaluation.

**Keywords:** Transformer models, Music, Generative, Sequence modelling, Attention mechanism

## 1 Introduction

A *riff* is a short, repeated motif or figure in the melody or accompaniment of a musical composition. Randel (1986) This type of composition is particularly prevalent in rock, metal, blues, funk and jazz. It is often the main feature that gives character and structure to a song. Recent developments in Transformers architecture have inspired many to generate sequences by treating musical notes as symbolic language.

Musical elements like notes, timestamps, and velocities are converted into discrete tokens similar to text processing. Suresh et al. (2020) A widely adopted method for representing musical notes digitally is the MIDI format, which encodes pitch, duration, and dynamics. Using such a format for musical expression has limitations like resolution and bandwidth constraints Shamooun (2024) when compared to richer representations like audio recordings and detailed scores. Modern guitar riffs frequently employ intense and abrupt expressive techniques, which contribute significantly to its overall identity. To accurately represent the nuanced expressive elements of modern music, a score-based format is preferable over simpler encodings like MIDI. A comprehensive list of these expressive elements is provided in Appendix A. Generating modern music demands capturing not only note patterns but also the associated expressions.

## 2 Encoding

The original Transformer model Vaswani et al. (2017) used a vocabulary of tokens derived from the input language corpus, with each token represented as an integer index. A note is represented by multiplying the guitar's fret number by the string number to generate a

unique index. This approach not only helps the model learn the patterns of movement on the fretboard but also provides flexibility to accommodate alternative tunings, which are increasingly common in modern guitar playing. With 6 strings and 23 frets per string, the instrument offers 138 distinct fretboard positions. The rhythmic structure is divided into seven beat durations: whole, 1/2, 1/4, 1/8, 1/16, and 1/32 and 1/64 notes. These divisions provide varying levels of temporal granularity for representing note durations. Each beat can further be subdivided into 14 distinct rhythmic values which is listed in Appendix B, which makes a total to 82 unique beat indices. Since palm muting is a prominent feature in metal music, it is assigned a unique token in the representation. This encoding process produces  $138 * 98 * 2 = 27048$  unique musical tokens. The expressive vocabulary listed in Appendix A is represented using the same format. A unique integer is assigned to various accents used in guitar playing. The necessity of using relative positional encodings rather than absolute ones, is already shown to enable state reuse without causing temporal confusion Dai et al. (2019a). Since music is highly sequential such an encoding type should preserve timing, order, and spacing between notes.

Let  $L$  be the sequence length and  $d$  be the dimensionality of each attention head.

We define the relative positional embedding matrix:

$$\mathbf{R} \in \mathbb{R}^{(2L-1) \times d}$$

where the size  $2L-1$  corresponds to all possible relative positions from  $-(L-1)$  to  $+(L-1)$ .

The embedding lookup for a relative position  $r = j - i$  (shifted by  $L-1$  to be non-negative) is given by:

$$\mathbf{r}(r) = \mathbf{R}[r + (L-1)], \quad r \in \{-(L-1), \dots, 0, \dots, (L-1)\}$$

Here,  $\mathbf{r}(r) \in \mathbb{R}^d$  is the embedding vector corresponding to relative position  $r$ .

## 2.1 Note representations

The formula for note encodings is given by:

$$note\_encodings = ni + bt * 276 + bb * 276 * 14 \quad (1)$$

$$ni = fn + sn * 23 + 6 * 23 * nval \quad (2)$$

$$nval = \begin{cases} 1 & \text{if, Palm Mute} \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

$$bt = \begin{cases} 1 + d & \text{if, Triplet(3)} \\ 2 + d & \text{if, Quintuplet(5)} \\ 3 + d & \text{if, Sextuplet(6)} \\ 4 + d & \text{if, Septuplet(7)} \\ 5 + d & \text{if, Nonuplet(9)} \\ 6 + d & \text{if, Undecuplet(11)} \end{cases} \quad (4)$$

$$d = \begin{cases} 7 & \text{if, Dotted note} \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

$$bb = \begin{cases} 0 & \text{if, 1 / 64} \\ 1 & \text{if, 1 / 32} \\ 2 & \text{if, 1 / 16} \\ 3 & \text{if, 1 / 8} \\ 4 & \text{if, 1 / 4} \\ 5 & \text{if, 1 / 2} \\ 6 & \text{if, 1 / 1} \end{cases} \quad (6)$$

## 2.2 Accent representations

### 2.2.1 CHORD

A chord is a group of notes played together, typically three or more. Figure 1 depicts a C major chord comprising of 3 notes - root note [C] played on the fret 2 / 5th string, major third [G] on fret 1 / 4th string and perfect fifth [E] on fret 2 / 1st string.

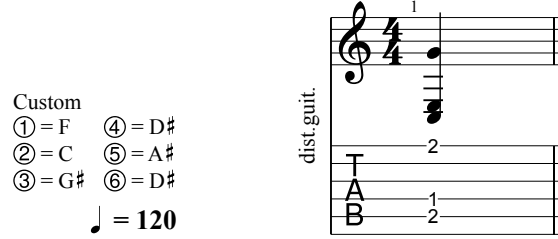


Figure 1: C major chord shape on a 6 string with custom tuning.

|       |            |       |            |       |            |
|-------|------------|-------|------------|-------|------------|
| 15413 | 27051      | 15389 | 27051      | 15321 | 27051      |
| C     | BARRE.NOTE | E     | BARRE.NOTE | G     | BARRE.NOTE |

Table 1: C major chord quarter note tokenisation in a sequence.

### 2.2.2 OTHER

Accentuations like BEND\_NOTE, SLIDE\_NOTE etc are represented by the note encodings from Section 2.1 followed by an accent token listed in Appendix A.

## 2.3 EOS and BAR

A music bar (also called a measure) is a fundamental unit in musical notation that divides music into segments of equal time, defined by the time signature. Each bar is separated by a BAR token. A collection of measures making a song is separated by an EOS token.

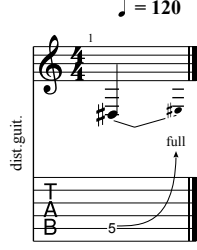


Figure 2: Fret 5 string 5 quarter note bend

|       |             |
|-------|-------------|
| 15416 | 27052       |
| C     | BEND_NOTE_1 |

Table 2: Bend note tokenisation

### 3 Embedding

A bar consists of a fixed number of beats, composed of musical notes that were tokenised as described in the previous section. Each bar is treated like a sentence. The tokens representing the accents or expression in riff are placed next to each note in the sentence. An example of a riff and its tokenisation is given in Figure 2. It has been demonstrated that the transformer equipped with relative attention is very well-suited for generative modelling of symbolic music. Huang et al. (2018). The chunking strategy used to carry over context between sequences during training follows the approach introduced in Yang et al. (2019), which builds upon Transformer-XL’s segment-level recurrence mechanism. This method enables the model to capture longer-range dependencies by reusing hidden states from previous token chunks across segments.

## 4 Training

### 4.1 Model

The Transformer is built from a decoder-only stack with masked self attention as described in Huang et al. (2018). Although, segment-level recurrence mechanism is captured in Transformer-XL Dai et al. (2019b), its memory is limited to one previous segment at a time. To model dependencies across a whole batch, the gaussian distributions of the sequences are learned through a Multi layer Perceptron layer. If the input is a batch of sequences defined by:

$$X \in \mathbb{R}^{B \times T \times d} \quad (7)$$

Where:

- $B$ : batch size (number of sequences)
- $T$ : number of tokens per sequence
- $d$ : embedding dimension
- $X_i \in \mathbb{R}^{T \times d}$ : the  $i$ -th sequence in the batch

Sequence level mean pooling is performed by:

$$\bar{x}_i = \frac{1}{T} \sum_{t=1}^T X_{i,t} \in \mathbb{R}^d \quad (8)$$

Gaussian parameters are then computed with a Multi Layer Perceptron. Let  $f : \mathbb{R}^d \rightarrow \mathbb{R}^{2d}$  be a learned MLP. Then:

$$[\mu_i, \log \sigma_i] = f(\bar{x}_i) \quad \text{with} \quad \mu_i, \log \sigma_i \in \mathbb{R}^d \quad (9)$$

Ensure positivity of standard deviation:

$$\sigma_i = \exp(\log \sigma_i) + \varepsilon \quad \text{where} \quad \varepsilon > 0 \quad (10)$$

Each sequence is then represented as a diagonal multivariate Gaussian:

$$\mathcal{N}_i = \mathcal{N}(\mu_i, \text{diag}(\sigma_i^2)) \quad (11)$$

Vectorized for the batch:

$$[\mu, \log \sigma] = f(\bar{X}) \in \mathbb{R}^{B \times 2d} \quad (12)$$

$$\sigma = \exp(\log \sigma) + \varepsilon \in \mathbb{R}^{B \times d} \quad (13)$$

The pairwise KL divergence between each pair of Gaussians Zhang et al. (2023):

$$\text{KL}(\mathcal{N}_i \parallel \mathcal{N}_j) = \frac{1}{2} \sum_{k=1}^d \left[ \log \left( \frac{\sigma_{j,k}^2}{\sigma_{i,k}^2} \right) + \frac{\sigma_{i,k}^2}{\sigma_{j,k}^2} + \frac{(\mu_{i,k} - \mu_{j,k})^2}{\sigma_{j,k}^2} - 1 \right] \quad (14)$$

This produces a KL matrix:

$$\text{KL\_matrix} \in \mathbb{R}^{B \times B}, \quad [\text{KL\_matrix}]_{i,j} = \text{KL}(\mathcal{N}_i \parallel \mathcal{N}_j) \quad (15)$$

A simpler version of the attention matrix from Dai et al. (2019a) is used from which the KL divergence-based sequence bias is subtracted:

$$\text{RA}_{\text{KL}} = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} + QR^\top - \alpha \cdot \text{KL}_{\text{bias}} \right) \quad (16)$$

A weighted sum over relative positional value contributions  $V_{\text{rel}}$  is then added:

$$\text{Output} = \text{RA}_{\text{KL}} \times V + \text{RA}_{\text{KL}} \times V_{\text{rel}} \quad (17)$$

## 4.2 Loss function

The model's loss is composed of three components:

$$\text{Totalloss} = \text{CrossEntropy} + \text{Repetitionloss} + \text{Sequenceloss} \quad (18)$$

### 4.2.1 CROSS ENTROPY LOSS

Cross-entropy loss is the standard choice for training deep neural networks and logistic regression models for both binary and multi-class classification tasks. ml glossary (2017)

#### 4.2.2 REPETITION LOSS

Repetition helps organise musical ideas, making pieces more coherent and memorable. It gives listeners reference points and expectations, aiding in the recognition and recall of themes or motifs. Hu et al. (2022) Composers have long used stochastic (probability-based) methods to inject randomness into repeated motifs, sometimes even employing quantum randomness for truly unpredictable results. Yamada et al. (2021) This demands careful consideration, as it relies on the listener’s inherent ability to recognise patterns and their natural inclination toward novelty. Since it is already known that tokens listed in Appendix A cannot repeat, it is penalised through repetition loss making them less likely to be selected again in subsequent steps. A variant of the repetition penalty introduced in Keskar et al. (2019) is employed; however, instead of greedy decoding, multinomial sampling is used to generate tokens to get semantically different output sequences Song et al. (2024). Including this loss during training eliminates the need to explicitly handle constraints for generating valid sequences at inference time

$$\text{Let } p_{b,t,v} = \text{softmax}\left(\frac{\text{logits}_{b,t,v}}{T}\right), \quad \text{for } b = 1, \dots, B; \ t = 0, \dots, T-1; \ v = 1, \dots, V \quad (19)$$

$$(20)$$

Compute average past probabilities:

$$\bar{p}_{b,t,v} = \frac{1}{t} \sum_{k=0}^{t-1} p_{b,k,v}, \quad \text{for } t \geq 1 \quad (21)$$

Repetition score (dot product of current and average previous probabilities)

$$R_{b,t} = \sum_{v=1}^V p_{b,t,v} \cdot \bar{p}_{b,t,v}, \quad \text{for } t \geq 1 \quad (22)$$

Final penalty (mean repetition score across batch and sequence, excluding)

$$\text{Repetitionloss} = \frac{\lambda}{B(T-1)} \sum_{b=1}^B \sum_{t=1}^{T-1} R_{b,t} \quad (23)$$

where  $\lambda$  is a repetition penalty weight hyper parameter.

#### 4.2.3 SEQUENCE PENALTY LOSS

Pairwise KL divergence between each pair of Gaussians is given by (14). Given a batch of  $B$  sequences, Sequence loss is defined by:

$$\mathcal{L}_{\text{KL}} = \frac{1}{B} \sum_{b=1}^B \text{KL}(\mathcal{N}_q^{(b)} \parallel \mathcal{N}_k^{(b)}) \quad (24)$$

To make the training more stable, the sequence loss is annealed and clamped.

$$\text{Sequenceloss} = \min\left(\frac{\text{step}}{\text{EPOCHS}} \cdot \left(\frac{\delta \cdot \mathcal{L}_{\text{KL}}}{\text{BATCH}}\right), 0.2\right) \quad (25)$$

where,  $\delta$  is a sequence weight hyper parameter.

#### 4.2.4 DATASET

Relevant statistics about the dataset used for training and evaluation are provided in Appendix (C)

### 5 Inference

Given that the model is autoregressive and expects an initial token, selecting the start token based on the distribution of starting notes of musical measures in the training data yields more consistent generation.

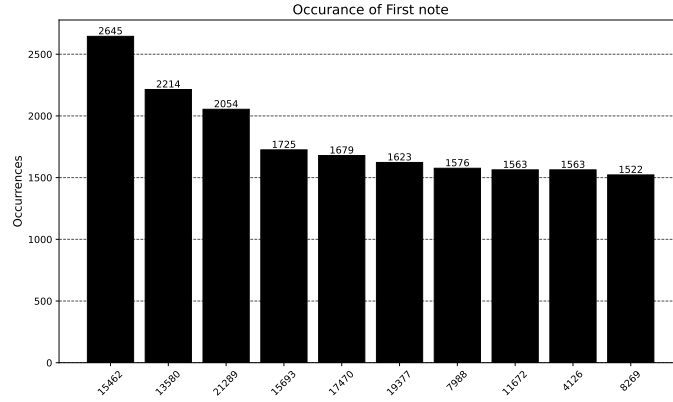


Figure 3: Token occurrences of first note of a musical measure in training data.

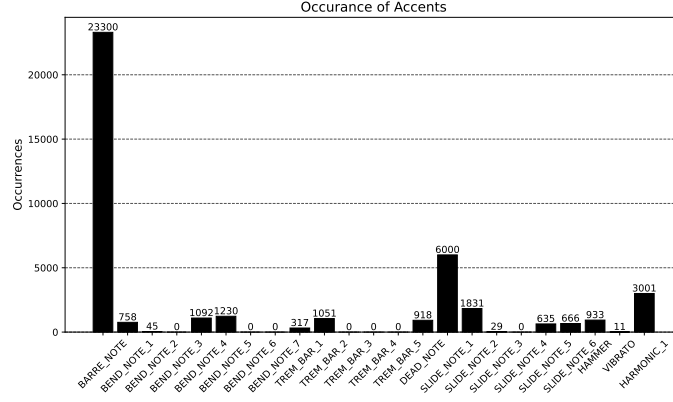


Figure 4: Accent occurrences in training data.

The idea of context carryover has been well-established in Dauphin et al. (2016). During inference, the model carries over context from previous sequences by using the last token of the preceding measure as the first token for the subsequent measure.

Loss values observed after 100 epochs of training:

| Loss          | Value  |
|---------------|--------|
| Cross Entropy | 1.4629 |
| Repetition    | 0.1368 |
| Sequence      | 0.1985 |
| Total         | 1.7983 |
| Validation    | 2.0622 |

Table 3: Loss values

To compare the quality of predicted tokens (P) from true distributions (Q), relative entropy is calculated using Kullback–Leibler divergence:

$$D_{KL}(P \parallel Q) = \sum_{x \in X} P(x) \log \left( \frac{P(x)}{Q(x)} \right) \quad (26)$$

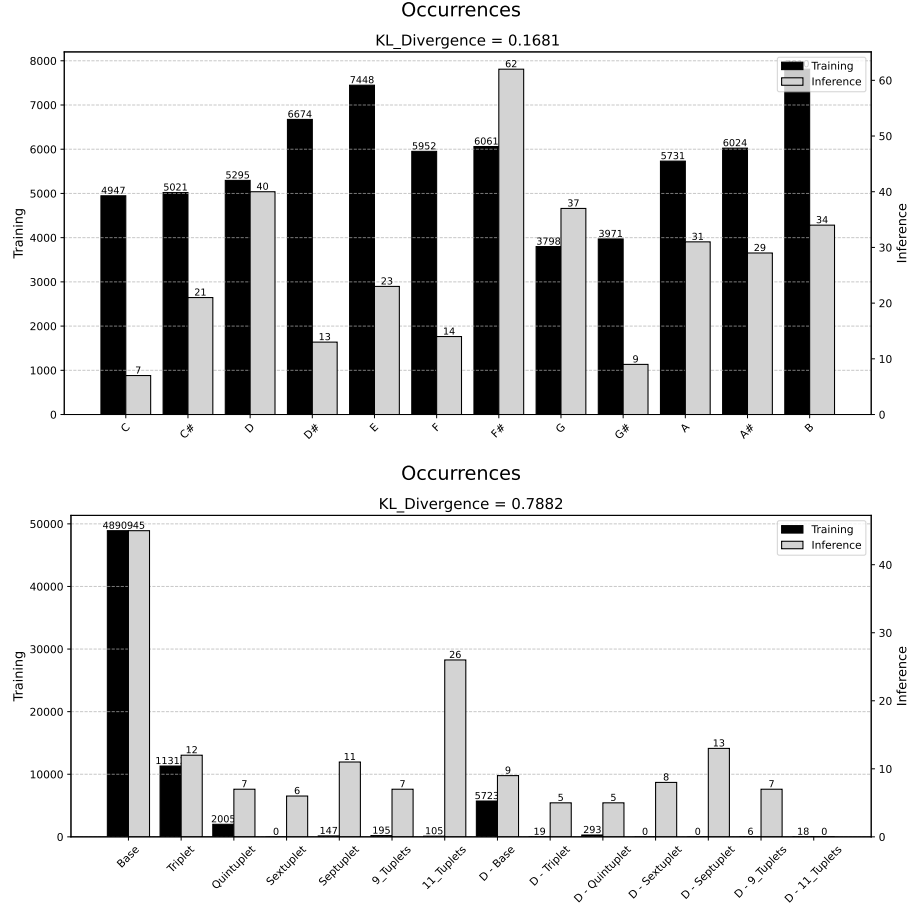


Figure 5: Note and beat occurrences of training data and test data

The model produces musically pleasing sequences, effectively combining notes, beats, and accents while maintaining sequence context between measures.

# TRANSFORMER RIFFAGE

Custom Tuning  
 ① = F    ④ = D#  
 ② = C    ⑤ = A#  
 ③ = G#   ⑥ = D#

Moderate ♩ = 120

dist. guitar

The musical score consists of four measures of music in 4/4 time, marked 'Moderate' with a tempo of 120 bpm. The guitar is tuned to a custom tuning: ① = F, ② = C, ③ = G#, ④ = D#, ⑤ = A#, ⑥ = D#. The score includes various guitar techniques such as palm mutes (P.M.), full bends (full), and slides (sl.). The first measure is marked '1' and the fourth measure is marked '4'. The score also includes a 'dist. guitar' label and a '32:1' ratio.

Figure 6: Test results showing 4 musical measures

Implementation details and hyper-parameter settings are provided in the associated GitHub repository Begari (2025).

## Acknowledgments and Disclosure of Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

**Appendix A.**

In this appendix a comprehensive list of accents played on the guitar is provided.

| <b>Accent</b> | <b>Unique Index</b> |
|---------------|---------------------|
| BAR           | 27049               |
| BOS           | 27050               |
| BARRE_NOTE    | 27051               |
| BEND_NOTE_1   | 27052               |
| BEND_NOTE_2   | 27053               |
| BEND_NOTE_3   | 27054               |
| BEND_NOTE_4   | 27055               |
| BEND_NOTE_5   | 27056               |
| BEND_NOTE_6   | 27057               |
| BEND_NOTE_7   | 27058               |
| TREM_BAR_1    | 27059               |
| TREM_BAR_2    | 27060               |
| TREM_BAR_3    | 27061               |
| TREM_BAR_4    | 27062               |
| TREM_BAR_5    | 27063               |
| DEAD_NOTE     | 27064               |
| SLIDE_NOTE_1  | 27065               |
| SLIDE_NOTE_2  | 27066               |
| SLIDE_NOTE_3  | 27068               |
| SLIDE_NOTE_4  | 27069               |
| SLIDE_NOTE_5  | 27070               |
| SLIDE_NOTE_6  | 27071               |
| HAMMER        | 27072               |
| VIBRATO       | 27073               |
| HARMONIC_1    | 27074               |
| EOS           | 27075               |

Table 4: Example of unique indexing of guitar accents.

## Appendix B.

In this appendix a comprehensive list of rhythmic patterns are listed.

| Pattern             | Unique Index |
|---------------------|--------------|
| Base                | 1            |
| Triplet             | 2            |
| Quintuplet          | 3            |
| sextuplet           | 4            |
| septuplet           | 5            |
| 9 tuplets           | 6            |
| 11-tuplets          | 7            |
| Dotted - Base       | 8            |
| Dotted - Triplet    | 9            |
| Dotted - Quintuplet | 10           |
| Dotted - sextuplet  | 11           |
| Dotted - septuplet  | 12           |
| Dotted - 9 tuplets  | 13           |
| Dotted - 11-tuplets | 14           |

Table 5: Example of unique indexing of rhythm.

## Appendix C.

This appendix explains relevant statistics about the dataset used for training and evaluation.

### C.1 Preparation

The rhythm guitar tracks were extracted manually from the original Guitar Pro files. Finger adjustments present in the source files are described in detail in the encoding section. No additional normalization or filtering—such as pitch standardization or sequence length filtering—was performed beyond this manual selection.

The dataset consists exclusively of rhythm guitar tracks in .gp5 format with a custom tuning (F C G# D# A# D#). It contains 6,208 sequences, each with a maximum length of 32 tokens.

An 80/20 split was used to create the training and validation sets. For testing, the top 10 first notes of each measure were selected as prompts for sequence generation and evaluation.

Due to copyright restrictions, the original .gp5 files and preprocessing scripts cannot be shared. However, the code and model pipeline are designed to operate on any Guitar Pro dataset, provided the files are preprocessed into the expected format as described.

## References

Vishakh Begari. Supervised composition and riff engine. <https://github.com/DjgentleViBe/SCORE>, 2025. Accessed: 2025-07-11.

- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G. Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *CoRR*, abs/1901.02860, 2019a. URL <http://arxiv.org/abs/1901.02860>.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G. Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *CoRR*, abs/1901.02860, 2019b. URL <http://arxiv.org/abs/1901.02860>.
- Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. *CoRR*, abs/1612.08083, 2016. URL <http://arxiv.org/abs/1612.08083>.
- Zhejing Hu, Xiao Ma, Yan Liu, Gong Chen, and Yongxu Liu. The beauty of repetition in machine composition scenarios. In *Proceedings of the 30th ACM International Conference on Multimedia*, MM '22, page 1223–1231. ACM, October 2022. doi: 10.1145/3503161.3548130. URL <http://dx.doi.org/10.1145/3503161.3548130>.
- Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, and Douglas Eck. An improved relative self-attention mechanism for transformer with application to music generation. *CoRR*, abs/1809.04281, 2018. URL <http://arxiv.org/abs/1809.04281>.
- Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. CTRL: A conditional transformer language model for controllable generation. *CoRR*, abs/1909.05858, 2019. URL <http://arxiv.org/abs/1909.05858>.
- ml glossary. Machine learning glossary. [https://ml-cheatsheet.readthedocs.io/en/latest/loss\\_functions.html](https://ml-cheatsheet.readthedocs.io/en/latest/loss_functions.html), 2017.
- Don Michael Randel. *New Harvard Dictionary of Music*. Harvard University Press., Cambridge, MA, 1986.
- Evan Shamoan. What’s wrong with midi? <https://www.perfectcircuit.com/signal/whats-wrong-with-midi>, 2024.
- Yifan Song, Guoyin Wang, Sujian Li, and Bill Yuchen Lin. The good, the bad, and the greedy: Evaluation of llms should not ignore non-determinism, 2024. URL <https://arxiv.org/abs/2407.10457>.
- Rahul Suresh, Aashiq Muhamed, Liang Li, Patrick Cavins, Maryam Rezapoor, Wayne Chi, and Suri Yaddanapudi. Using transformers to create music in aws deep-composer music studio. <https://aws.amazon.com/blogs/machine-learning/using-transformers-to-create-music-in-aws-deepcomposer-music-studio/>, 2020.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL <http://arxiv.org/abs/1706.03762>.

- Reiko Yamada, Samuele Grandi, Gorka Muñoz-Gil, Luca Barbiero, Albert Aloy, and Maciej Lewenstein. Applications of quantum randomness: From rabi oscillations to fourier axis controlling the musical timbre. *International Journal of Music Science, Technology and Art*, 3(2):17–25, August 2021. ISSN 2612-2146. doi: 10.48293/ijmsta-77. URL <https://doi.org/10.48293/ijmsta-77>.
- Zhilin Yang, Zihang Dai, Yiming Yang, Jaime G. Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. Xlnet: Generalized autoregressive pretraining for language understanding. *CoRR*, abs/1906.08237, 2019. URL <http://arxiv.org/abs/1906.08237>.
- Yufeng Zhang, Jialu Pan, Li Ken Li, Wanwei Liu, Zhenbang Chen, Xinwang Liu, and J Wang. On the properties of kullback-leibler divergence between multivariate gaussian distributions. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 58152–58165. Curran Associates, Inc., 2023. URL [https://proceedings.neurips.cc/paper\\_files/paper/2023/file/b5b4d92374323c53c24bbbc8ee0e715c-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2023/file/b5b4d92374323c53c24bbbc8ee0e715c-Paper-Conference.pdf).